

Hermes AI Operating System - Architecture Design Document

Why the redesign

Old architecture was a linear pipeline (Scout -> Validator -> Publisher). It tightly coupled agents, making it difficult to add new capabilities. The new architecture is event-driven where agents never call each other directly.

Core Principles

- Agents communicate only through events.
- Hermes orchestrates; agents execute.
- MCPs abstract data sources.
- Providers abstract AI models.
- Shared memory stores state.
- Slack is the control plane.

Proposed Project Structure

```
Internship-AI/  
■■■ agents/  
■ ■■■ scout/  
■ ■■■ validator/  
■ ■■■ publisher/  
■ ■■■ ranking/  
■ ■■■ salary/  
■ ■■■ duplicate/  
■ ■■■ analytics/  
■■■ core/  
■ ■■■ event_bus.py  
■ ■■■ orchestrator.py  
■ ■■■ registry.py  
■ ■■■ dispatcher.py  
■ ■■■ lifecycle.py  
■■■ providers/  
■ ■■■ openai_provider.py  
■ ■■■ claude_provider.py  
■ ■■■ gemini_provider.py  
■■■ mcps/  
■ ■■■ gateway.py  
■ ■■■ careers.py  
■ ■■■ greenhouse.py  
■ ■■■ lever.py  
■ ■■■ ashby.py  
■■■ memory/  
■■■ services/  
■■■ slack/  
■■■ commands/  
■■■ config/  
■■■ prompts/  
■■■ logs/  
■■■ data/  
■■■ main.py
```

Workflow

Slack -> Hermes -> Dispatcher -> Event Bus -> Independent Agents -> Shared Memory -> Publisher -> Slack

Planned Improvements

1. Event-driven runtime
2. Capability registry
3. Dynamic agent registration
4. Shared memory
5. Async execution
6. AI provider abstraction
7. MCP gateway
8. Retry policies
9. Metrics & logging
10. Scheduler
11. Slack mission control
12. Future support for CRM, Sales, Recruitment and Research agents.

Development Roadmap

- Phase 1: Core runtime
- Phase 2: AI providers
- Phase 3: MCP gateway
- Phase 4: Agent implementations
- Phase 5: Slack UI
- Phase 6: Testing, metrics and deployment